EXERCÍCIO PROPOSTO 1

Modele ALUNO: ALUNO(id_aluno, matricula, nome, email, nascimento, coeficiente, bolsista, observacoes)

Escolha um tipo MySQL para cada atributo: id_aluno INT AUTO_INCREMENT / matricula CHAR(10) / nome VARCHAR (120) / email VARCHAR (254) / nascimento DATE / coeficiente DECIMAL (p,s) / bolsista BOOLEAN / observações TEXT

Defina comprimentos e precisão com justificativas: matricula CHAR(10) código de tamanho fixo e preserva zeros/ nome VARCHAR(120) texto curto e variável / email VARCHAR(254) texto curto e variável / coeficiente DECIMAL(p,s) deve ter precisão definida conforme a regra do sistema / observacoes TEXT indicado para conteúdo livre e mais extenso

EXERCÍCIO PROPOSTO 2

```
CREATE TABLE reserva (
  id_reserva INT NOT NULL AUTO_INCREMENT,
  id_sala INT NOT NULL,
  id_responsavel INT NOT NULL,
  inicio DATETIME NOT NULL,
  fim DATETIME NOT NULL,
  situacao VARCHAR(20) NOT NULL DEFAULT 'ATIVA',

  CONSTRAINT pk_reserva
    PRIMARY KEY (id_reserva),

  CONSTRAINT fk_reserva_sala
    FOREIGN KEY (id_sala)
    REFERENCES sala(id_sala),

  CONSTRAINT fk_reserva_responsavel
    FOREIGN KEY (id_responsavel)
    REFERENCES responsavel(id_responsavel),

  CONSTRAINT ck_reserva_periodo
    CHECK (fim > inicio),
```

```
  CONSTRAINT ck_reserva_situacao
    CHECK (situacao IN ('ATIVA', 'CANCELADA', 'CONCLUIDA')),

  CONSTRAINT uq_reserva_sala_inicio
    UNIQUE (id_sala, inicio)
);
```

EXERCÍCIO PROPOSTO 3

DEPARTAMENTO → EMPREGADO: use ON DELETE SET NULL, pois o filho continua existindo

PROJETO → TAREFA: ON DELETE CASCADE, porque ao excluir o projeto, suas tarefas também devem ser excluídas

AUTOR → LIVRO_AUTOR: ON DELETE RESTRICT, impedindo excluir o autor enquanto houver registros relacionados em LIVRO_AUTOR

EXERCÍCIO PROPOSTO 4

Crie um INSERT que viole PRIMARY KEY: tentar inserir dois registros com a mesma chave primária

Crie um INSERT que viole UNIQUE e outro que viole CHECK: tentar inserir dois registros com o mesmo valor em um campo que deve ser único, como por exemplo o e-mail/ CHECK: tentar inserir um valor fora da regra definida

Tente inserir uma FK órfã: tentar cadastrar um registro filho apontando para um registro pai que não existe

Teste uma exclusão bloqueada por RESTRICT: tentar excluir um registro pai que ainda possui registros filhos

Teste uma exclusão propagada por CASCADE e descreva o estado final esperado: ao excluir o registro pai, os filhos relacionados também devem ser excluídos automaticamente. É esperado que o pai e seus filhos dependentes não existam mais

EXERCÍCIO PROPOSTO 5

Identifique os fatos distintos misturados: ele mistura dados do cliente, venda e produto

Demonstre uma anomalia de atualização, uma de inserção e uma de exclusão: Atualização: se o nome de um cliente mudar, seria necessário alterar esse nome em todas as vendas em que ele aparece

Inserção: se os dados do produto estiverem misturados com a venda, pode ser impossível cadastrar um produto antes de existir uma venda relacionada

Anomalia de exclusão: ao excluir a última venda ou o último item relacionado a um produto, pode-se apagar também a única informação existente sobre esse produto

Decomponha a relação em tabelas com PKs e FKs: CLIENTE (id_cliente PK, nome_cliente)

VENDA (id_venda PK, data, id_cliente FK)

PRODUTO (id_produto PK, nome_produto)

ITEM_VENDA (id_venda PK/FK, id_produto PK/FK, quantidade, preco_praticado)

Indique qual atributo histórico deve permanecer no item da venda: quantidade

EXERCÍCIO • DESAFIO INTEGRADOR

```
CREATE DATABASE IF NOT EXISTS clinica
CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci;
USE clinica;

CREATE TABLE paciente (
  id_paciente INT NOT NULL AUTO_INCREMENT,
  nome VARCHAR(120) NOT NULL,
  email VARCHAR(254) NOT NULL,

  CONSTRAINT pk_paciente PRIMARY KEY (id_paciente),
  CONSTRAINT uq_paciente_email UNIQUE (email)
);

CREATE TABLE medico (
  id_medico INT NOT NULL AUTO_INCREMENT,
  nome VARCHAR(120) NOT NULL,
  registro VARCHAR(30) NOT NULL,

  CONSTRAINT pk_medico PRIMARY KEY (id_medico),
  CONSTRAINT uq_medico_registro UNIQUE (registro)
);

CREATE TABLE consulta (
  id_consulta INT NOT NULL AUTO_INCREMENT,
  id_paciente INT NOT NULL,
  id_medico INT NOT NULL,
  data_hora DATETIME NOT NULL,
  estado VARCHAR(20) NOT NULL DEFAULT 'AGENDADA',

  CONSTRAINT pk_consulta PRIMARY KEY (id_consulta),

  CONSTRAINT ck_consulta_estado
    CHECK (estado IN ('AGENDADA', 'REALIZADA', 'CANCELADA')),

  CONSTRAINT fk_consulta_paciente
    FOREIGN KEY (id_paciente)
    REFERENCES paciente(id_paciente)
    ON DELETE RESTRICT
    ON UPDATE RESTRICT,

  CONSTRAINT fk_consulta_medico
    FOREIGN KEY (id_medico)
    REFERENCES medico(id_medico)
    ON DELETE RESTRICT
    ON UPDATE RESTRICT
);
```

```
CREATE TABLE procedimento (
  id_procedimento INT NOT NULL AUTO_INCREMENT,
  nome VARCHAR(120) NOT NULL,
  valor DECIMAL(10,2) NOT NULL,

  CONSTRAINT pk_procedimento PRIMARY KEY (id_procedimento),
  CONSTRAINT ck_procedimento_valor CHECK (valor >= 0)
);

CREATE TABLE consulta_procedimento (
  id_consulta INT NOT NULL,
  id_procedimento INT NOT NULL,

  CONSTRAINT pk_consulta_procedimento
    PRIMARY KEY (id_consulta, id_procedimento),

  CONSTRAINT fk_cp_consulta
    FOREIGN KEY (id_consulta)
    REFERENCES consulta(id_consulta)
    ON DELETE CASCADE
    ON UPDATE RESTRICT,

  CONSTRAINT fk_cp_procedimento
    FOREIGN KEY (id_procedimento)
    REFERENCES procedimento(id_procedimento)
    ON DELETE RESTRICT
    ON UPDATE RESTRICT
);
```